Software Seguro

```
root@software-seguro:~# ./start_course.sh
```

Clase 8: Vulnerabilidades de Inyección (SQLi)

Curso Profesional de Ciberseguridad Web y Ethical Hacking

Repaso Clase 7

Venta de autos - El analista

Objetivos de la Sesión

Comprender las consecuencias de una falta de sanitización en las entradas que procesa el servidor, permitiendo alterar la base de datos.

Qué es SQL Injection

Vulnerabilidad de seguridad web que ocurre cuando datos ingresados por un usuario se concatenan directamente en una consulta a la base de datos sin la debida validación o escape, permitiendo al atacante manipular la estructura de dicha consulta.

Esto permite a un tercero interferir en las consultas que una aplicación realiza a su motor de base de datos relacional (como PostgreSQL, MySQL, SQL Server u Oracle), pudiendo leer datos sensibles, modificarlos, destruirlos o, en casos extremos, ejecutar comandos en el sistema operativo del servidor.



Ejemplo de consulta vulnerable

Consulta : SELECT * FROM users u WHERE u.username = '$user' AND u.password = '$password'

Payload : admin' --

Consulta final : SELECT * FROM users u WHERE u.username = 'admin' --' AND u.password = '$password'

Candidatos

Premisa: “La aplicación arma una consulta SQL concatenando directamente el valor que mandó el usuario, en vez de tratarlo como un dato separado.”

Para identificar candidatos prioritarios a SQL Injection, el criterio clave es rastrear cualquier dato de entrada que termine formando parte de la estructura de una sentencia SQL en el servidor, sin importar si la página muestra los datos, si los guarda o si devuelve una pantalla vacía.

Candidatos

- **Buscadores de texto libre (**?q=, ?search=)**:** Suelen implementar el operador LIKE '%...%' para búsquedas parciales. Ejemplo vulnerable en backend: WHERE nombre LIKE '%" + search + "%'
- **Filtros por categoría, estado o rango** (?cat=, ?status=, ?precio_max=): Al ser valores que en el frontend provienen de menús desplegables (select), casillas de verificación o botones de opción, suele asumirse erróneamente que "un menú fijo no puede enviar texto malicioso" Ejemplo vulnerable en backend: WHERE categoria_id = " + req.query.cat
- **Ordenamiento de resultados** (?orderby=, ?sort=, ?dir=): Las sentencias preparadas y consultas parametrizadas estándar no admiten nombres de columnas ni modificadores (ASC, DESC) como parámetros. Ejemplo vulnerable en backend: ORDER BY " + req.query.orderby + " " + req.query.dir
- **Formularios de autenticación (Login y recuperación de cuenta)**: Es el objetivo más atacado porque un bypass aquí permite saltar el control de acceso completo sin conocer contraseñas válidas, mediante cláusulas siempre verdaderas (tautologías). Ejemplo vulnerable en backend: WHERE email = '" + email + "' AND password = MD5('" + pass + "')

- **Identificadores de recursos en URLs o rutas REST** (/api/items/45, ?id=45): Representan claves primarias numéricas. Con frecuencia el backend las pasa tal cual a la consulta sin comillas: Ejemplo vulnerable en backend: WHERE id = " + req.params.id
- **Encabezados HTTP almacenados en base de datos** (User-Agent, X-Forwarded-For, Referer): AMuy común en sistemas de estadísticas, trazabilidad o registro de auditoría donde se guardan metadatos de la petición mediante sentencias INSERT. Al no venir de un formulario visible, se suele asumir falsamente que no son manipulables por un atacante. Ejemplo vulnerable en backend: INSERT INTO logs (ip, user_agent) VALUES ('" + ip + "', '" + ua + "')
- **Manejo de sesiones y cookies basadas en base de datos:** Cuando la aplicación valida tokens de sesión, estados de carritos de compras o preferencias guardadas en cookies directamente contra tablas de la base de datos sin parametrizar. Ejemplo vulnerable en backend: WHERE session_token = '" + cookies.session + "'
- **Paginación** (?limit=, ?offset=, ?page=): De manera similar a los identificadores, se asume que siempre son enteros inofensivos. Al no validarse que sean números estrictos antes de concatenarlos en las cláusulas LIMIT u OFFSET, se convierten en puntos de inyección ciega (Blind SQLi). Ejemplo vulnerable en backend: LIMIT " + req.query.limit + " OFFSET " + req.query.offset

Cómo detectarlo

Una vez tenemos los candidatos pasamos a realizar algunas pruebas de concepto PoC

- Mandar caracteres con significado especial en SQL: comilla simple ('), comilla doble ("), punto y coma (;), guiones (--), etc.
  - Señales de que hay algo: error 500, mensaje de error de base de datos expuesto en pantalla, cambio de comportamiento respecto a un input normal, o una respuesta que tarda distinto.
  - Si el error de la base de datos se muestra tal cual, ya es información valiosa: motor de base de datos, a veces la consulta original.

In-band SQLi (Inyección en banda o clásica)

Es la más directa. El atacante usa el mismo canal para enviar el ataque y ver los resultados reflejados inmediatamente en la pantalla.

- **Basada en UNION (UNION-based):** Aprovecha el operador UNION de SQL para "pegar" el resultado de otra consulta debajo de los resultados normales que ya muestra la aplicación.
  - Ejemplo: En un catálogo de productos que muestra SELECT nombre, precio FROM productos WHERE categoria = 'libros':
  - Entrada del atacante: ' UNION SELECT usuario, password FROM usuarios –
  - Resultado: La lista de productos en pantalla ahora incluye también los nombres y contraseñas de la tabla de usuarios.

- **Basada en errores (Error-based):** Provoca deliberadamente un fallo en la base de datos para que el motor devuelva un mensaje de error detallado que contenga la información buscada.
  - Ejemplo: En un buscador que muestra los errores de base de datos en pantalla:
  - Entrada del atacante: password FR' AND 1=CAST((SELECTOM usuarios WHERE id=1) AS INT) –
  - Resultado: El motor intenta convertir un texto (la contraseña) a número entero y falla, mostrando en la página:

SQL Injection basado en UNION

El operador UNION permite adjuntar los resultados de una consulta propia a la consulta legítima que hace la aplicación.

Para que una consulta con UNION sea válida y no lance error, el motor SQL impone dos reglas estrictas:

- Ambas consultas deben devolver exactamente la misma cantidad de columnas.
- Los tipos de datos de las columnas correspondientes deben ser compatibles (texto con texto, número con número).

Supongamos que el parámetro vulnerable es un filtro de categorías: ?cat=1. El atacante necesita saber cuántas columnas pide el backend originalmente (SELECT col1, col2, col3 FROM productos WHERE cat=1).

Técnica 1: Inyectar ORDER BY incremental

?cat=1 ORDER BY 1-- (La página carga normal)
?cat=1 ORDER BY 2-- (La página carga normal)
?cat=1 ORDER BY 3-- (La página carga normal)
?cat=1 ORDER BY 4-- (La página da error o se rompe")

Técnica 2: Inyectar UNION SELECT NULL

?cat=1 UNION SELECT NULL-- (Error: number of columns doesn't match)
?cat=1 UNION SELECT NULL, NULL-- (Error: number of columns doesn't match)
?cat=1 UNION SELECT NULL, NULL, NULL-- (Carga con éxito)

Blind SQLi: cuando no se ve nada

La aplicación no muestra datos de la base ni mensajes de error en pantalla. Parece invulnerable a simple vista, pero el atacante deduce la información haciendo preguntas de "verdadero o falso" (pregunta por pregunta, letra por letra).

- **Basada en booleanos (Boolean-based):** Hace una pregunta lógica. Si la respuesta es verdadera, la página carga normal (o muestra un producto); si es falsa, la página carga vacía o con un mensaje genérico de error.
  - Mandar una condición verdadera o falsa (' AND 1=1 -- vs ' AND 1=2 --) y comparar si la respuesta cambia (aparece contenido / no aparece, cambia el tamaño).

- **Basada en tiempo (Time-based):** En palabras simples: Como la página siempre responde igual visualmente (sea verdadero o falso), el atacante le ordena a la base de datos que "se duerma" unos segundos únicamente si su suposición es correcta. La respuesta se mide con un cronómetro.
  - Forzar un delay condicional (' AND IF(1=1, SLEEP(5), 0) --) y medir el tiempo de respuesta.

Out-of-band SQLi

Investigar de tarea

Diferencias entre dbms

Oracle	`--comment`
Microsoft	`--comment` `/*comment*/`
PostgreSQL	`--comment` `/*comment*/`

MySQL	`#comment`
	`-- comment` [Note the space after the double dash]
	`/*comment*/`

Oracle	`SELECT banner FROM v$version` `SELECT version FROM v$instance`
Microsoft	`SELECT @@version`
PostgreSQL	`SELECT version()`
MySQL	`SELECT @@version`

El rol de SQLmap

Herramienta de código abierto diseñada para automatizar la detección y evaluación de vulnerabilidades de inyección SQL. En auditorías de seguridad y pruebas de penetración autorizadas, cumple el rol de verificar si los puntos de entrada de una aplicación interactúan de forma insegura con la base de datos subyacente.

- **Identifica el DBMS:** Determina el motor (MySQL, PostgreSQL, Oracle, MSSQL, SQLite, etc.) analizando dialectos de error o respuestas diferenciales.
- **Prueba técnicas de inyección:** Inyecta secuencias de prueba en los parámetros indicados para medir si provocan cambios lógicos en la respuesta (TRUE vs FALSE), errores sintácticos o retardos de tiempo medibles.
- **Mide la estabilidad de la respuesta:** Compara el tamaño en bytes, código HTTP y estructura del DOM de las respuestas para descartar falsos positivos causados por contenido dinámico de la web.

Parámetros y Flags más comunes

Software Seguro

- **Definición de objetivo:**
  - -u <URL>: Especifica la URL objetivo a evaluar.
  - -r <archivo_request.txt>: Carga una petición HTTP completa interceptada (por ejemplo, desde un proxy como Burp Suite o OWASP ZAP), evaluando headers, cookies y cuerpo POST.
- **Configuración de prueba:**
  - --level (1-5): Define la profundidad de las pruebas. El nivel 1 prueba parámetros GET y POST. Niveles más altos (2 a 5) incluyen la evaluación de encabezados HTTP (Cookie, User-Agent, Referer).
  - --risk (1-3): Controla el tipo de pruebas. El riesgo 1 es seguro y no intrusivo; niveles superiores ejecutan pruebas basadas en UPDATE o subconsultas más pesadas que podrían degradar el rendimiento o alterar datos.
- **Técnicas de inyección:**
  - --technique=<B/E/U/S/T/Q>: Permite restringir el tipo de prueba a evaluar:
  - B: Boolean-based blind. E: Error-based.
  - U: UNION query-based. S: Stacked queries.T: Time-based blind. Q: Inline queries.
- **Detección y optimización:**
  - --dbms=<nombre>: Especifica directamente el motor de base de datos para omitir la fase de detección inicial y acelerar el análisis.
  - --threads=<n>: Define la cantidad de peticiones concurrentes para agilizar pruebas inferenciales (booleanas o de tiempo).

¿Usar un ORM o Stored Procedures me hace inmune a SQLi?

Usar un ORM o procedimientos almacenados (SP) no elimina por sí solo el riesgo de SQL Injection porque la vulnerabilidad nace de la concatenación de texto, sin importar dónde ocurra: un ORM se vuelve vulnerable en cuanto se recurre a consultas crudas (raw queries), fragmentos literales sin sanitizar o cláusulas no parametrizables como ORDER BY, mientras que un SP es igualmente explotable si en su interior ejecuta SQL dinámico concatenando parámetros (por ejemplo, con EXEC() o sp_executesql) en lugar de vincularlos como valores fijos. La única defensa real es la parametrización estricta (separar código de datos) combinada con listas blancas para identificadores dinámicos.

Prevencion y Mitigacion de SQLi

- **Consultas preparadas (Sentencias parametrizadas):** La defensa obligatoria y principal; separa estrictamente la estructura SQL de los datos del usuario.
- **Validación por lista blanca (whitelisting):** Imprescindible para nombres de tablas, columnas y cláusulas (ORDER BY, ASC/DESC), que no aceptan parámetros.
- **Principio de menor privilegio:** Configurar la cuenta de base de datos con los permisos mínimos necesarios (sin privilegios de administrador ni comandos destructivos como DROP).
- Tipado estricto y validación de entrada: Forzar que enteros, booleanos o UUIDs sean validados en el backend antes de llegar a la capa de datos.
- **Ocultar errores en producción:** Desactivar mensajes detallados de la base de datos de cara al usuario para bloquear ataques basados en errores (Error-based).
- **Firewall de aplicaciones web (WAF):** Capa complementaria de detección para bloquear patrones maliciosos conocidos antes de que impacten en el servidor.

Labs

Asignación para el Hogar

Resolver y enviar los objetivos del Módulo III:

- Ingresar a la plataforma de Software Seguro, en las opciones les aparecera el curso “Curso Profesional de Ciberseguridad Web y Ethical Hacking”, dentro del curso tienen los distintos módulos con sus objetivos que deberán enviar antes de la próxima sesión

¿Preguntas?

Fin de la Sesión 5.

https://softwareseguro.com.ar